

CS232L Operating Systems Lab

Lab 05: Introduction to C Programming

CS Program
Habib University

Fall 2019

1 Introduction

In this lab you will learn how to:

1. use aggregate types (arrays, structs) in C
2. use dynamic memory allocation in C
3. use string manipulation functions in C

2 Aggregate types

2.1 Arrays

Arrays let you aggregate multiple objects of the same type together. The standard guarantees that these aggregated objects would be placed contiguously in memory facilitating access via loops.

Arrays can be declared by placing a pair of [], with an optional size in between, after the identifier in a declaration.

Arrays are collections of objects. They cannot be assigned to, compared, or accessed in their entirety. Instead one has to access each element of an array individually and perform operations on them one by one in order to perform the operation on the whole array. Individual array elements can be accessed by specifying the array identifier followed by [] containing an integer designating the 0-indexed position of the array element in question.

The following code declares, initializes - using designated initializers, and prints an array of 5 integers.

```
1 #include <stdio.h>
2 int main()
3 {
4     int arr[5] = {[0]=1, [1]=33, [4]=55};
5     for (int i=0; i < 5; i++){
6         printf("arr[i]=%d\n", arr[i]);
7     }
8     return 0;
9 }
```

Listing 1: Arrays in C

It is the programmer's responsibility to access the valid elements of the array i.e. C does not provide protection against out-of-bound array access - when the index value is below the lower limit or above the upper limit.

Since C99 arrays can be fixed length arrays (FLAs) or variable length arrays (VLAs). The size of the FLAs is known at compile time and should be specified via an integer constant expression

(ICE) during their declaration. For FLAs whose size is not specified during their declarations, the compiler will try to deduce their size from their initializers. All expressions below declare two FLAs of 4 doubles.

```
1 double D[4];
2 double E[] = { [3] = 42.0 , [2] = 37.0 , };
3 double F[] = { 22.0 , 17.0 , 1, 0.5 , };
```

Listing 2: FLAs

VLAs on the other hand can have sizes which are not known at compile time but they come with the restrictions that they cannot have initializers and they cannot be declared outside of functions.

The length of an array can be calculated by the expression: $(\text{sizeof } A) / (\text{sizeof } A[0])$

Since array objects don't have values, i.e., they cannot be assigned to or manipulated directly as an ensemble, arrays cannot be passed as function parameters by value. Instead they are passed as references.

When arrays are passed as function parameters, they **decay** to pointers i.e. the information about their size is lost. Therefore the above expression to calculate the length of the array will not work if used inside a function on array which has been passed as a parameter to the function.

```
1 #include <stdio.h>
2 void swap_double (double a[static 2]){
3     double tmp = a[0];
4     a[0] = a[1];
5     a[1] = tmp ;
6 }
7 int main (void){
8     double A[2] = { 1.0, 2.0, };
9     swap_double (A);
10    printf ("A[0] = %g, A[1] = %g\n", A[0] , A [1]) ;
11 }
```

Listing 3: Helloworld in C

In the listing above `swap_double()` acts on the original array passed as reference and not on a copy of it.

The `static` keyword inside array informs the compiler that the passed array would always have atleast 2 members. This information lets the compiler assume that the passed pointer can never be NULL and perform optimizations based on this assumption. It is, of course, the responsibility of the programmer that the every function call will respect the promise that they have made to the compiler.

Strings, in C, are 0-terminated arrays of characters. They are special in the sense that they have literals which can be used to initialize them.

```
1 // these 4 declarations are equivalent (strings)
2 char jay0 [] = "jay ";
3 char jay1 [] = { "jay " };
4 char jay2 [] = { 'j' , 'a' , 'y' , 0, };
5 char jay3 [4] = { 'j' , 'a' , 'y' , };
6
7
8 // these 2 declarations are equivalent (arrays of characters)
9 char jay4 [3] = { 'j' , 'a' , 'y' , };
10 char jay5 [3] = "jay";
```

Listing 4: Helloworld in C

C standard library provides special functions to deal with character arrays (`memXXX`) and strings (`strXXX`). Their declarations can be found in `string.h`.

Functions that operate on char arrays are as follows:

- `memcpy(target, source, len)` can be used to copy one array to another. These have to be known to be distinct arrays. The number of characters to be copied must be given as a third argument `len`.
- `memcmp(s0, s1, len)` compares two arrays in lexicographic order. That is, it first scans the initial segments of the two arrays that happen to be equal and then returns the difference between the two first characters that are distinct. If no differing elements are found up to `len`, 0 is returned.
- `memchr(s, c, len)` searches array `s` for the appearance of character `c`.

Next are the string functions:

- `strlen(s)` returns the length of the string `s`. This is simply the position of the first 0 character and not the length of the array. It is your duty to ensure that `s` is indeed a string: that it is 0-terminated.
- `strcpy(target, source)` works similarly to `memcpy`. It only copies up to the string length of the source, and therefore it doesn't need a `len` parameter. Again, source must be 0-terminated. Also, target must be big enough to hold the copy.
- `strcmp(s0, s1)` compares two arrays in lexicographic order, similarly to `memcmp`, but may not take some language specialties into account. The comparison stops at the first 0 character that is encountered in either `s0` or `s1`. Again, both parameters have to be 0-terminated.
- `strcoll(s0, s1)` compares two arrays in lexicographic order, respecting languagespecific environment settings. We will learn how to properly set this in subsection 8.6.
- `strchr(s, c)` is similar to `memchr`, only the string `s` must be 0-terminated.
- `strspn(s0, s1)` returns the length of the initial segment in `s0` that consist of characters that also appear in `s1`.
- `strcspn(s0, s1)` returns the length of the initial segment in `s0` that consist of characters that do not appear in `s1`.

```

1 1 # include <string .h>
2 2 # include <stdio .h>
3 3 int main ( int argc , char * argv [ argc +1]) {
4 4     size_t const len = strlen ( argv [0]) ; // Computes the length
5 5     char name [ len +1]; // Creates a VLA
6 6     // Ensures a place for 0
7 7     memcpy (name , argv [0] , len); // Copies the name
8 8     name [ len] = 0; // Ensures a 0 character
9 9     if (! strcmp (name , argv [0]) ) {
10 10 printf (" program name \"%s\" successfully copied \n",
11 11 name );
12 12 } else {
13 13 printf (" copying %s leads to different string %s\n",
14 14 argv [0] , name );
15 15 }
16 16 }
```

Listing 5: Modern C: LISTING 6.2. Using some of the string functions

2.2 Structures

The `struct` keyword in C lets you define new data types which aggregate objects of, unlike arrays, different base types.

Once you have composed a struct, it behaves as a new type and you can declare and define objects of this new type.

The listing below first makes a new type i.e. `struct tm` and then defines and initializes an object, `today`, of this new type. Values of a specific members of an object can be accessed by the use of the `.` operator between the object identifier and its member identifier.

```

1 struct tm {
2     int tm_sec ; // Seconds after the minute [0, 60]
3     int tm_min ; // Minutes after the hour [0, 59]
4     int tm_hour ; // Hours since midnight [0, 23]
5     int tm_mday ; // Day of the month [1, 31]
6     int tm_mon ; // Months since January [0, 11]
7     int tm_year ; // Years since 1900
8     int tm_wday ; // Days since Sunday [0, 6]
9     int tm_yday ; // Days since January [0, 365]
10    int tm_isdst ; // Daylight Saving Time flag
11 };
12
13 struct tm today = {
14     .tm_year = 2014 ,
15     .tm_mon = 2,
16     .tm_mday = 29,
17     .tm_hour = 16,
18     .tm_min = 7,
19     .tm_sec = 5,
20 };
21
22 printf ( " this year is %d, next year will be %d\n" ,
23         today.tm_year , today.tm_year +1) ;

```

Listing 6: Caption

Structs define a new type. Due to historical reasons whenever we want to declare an object of this new type we have to precede the typename with the keyword `struct` as shown below:

```

1 struct my_new_struct var;

```

Listing 7: struct

This can be cumbersome.

There is a general tool that can help us avoid that, by giving a symbolic name to an otherwise existing type: `typedef`. Using it, a type can have several names, and we can even reuse the tag name `C` that we used in the structure declaration

```

1 typedef struct my_new_struct my_new_struct;
2 typedef struct my_new_struct newstruct;

```

Listing 8: typedef

This way `struct my_new_struct`, `my_new_struct`, `newstruct` can all now be used interchangeably.

3 Dynamic memory allocation

C provides functions to allocate memory from the `heap` dynamically during run time. Some of them are:

```

1 #include <stdlib .h>
2 void * malloc ( size_t size );
3 void free ( void * ptr );
4 void * calloc ( size_t nmemb , size_t size );
5 void * realloc ( void * ptr , size_t size );
6 void * aligned_alloc ( size_t alignment , size_t size );

```

Listing 9: Heap functions

The caller passes the size of memory they want to allocate (in bytes) to `malloc` and `malloc`, if successful, would return a pointer to the start of memory segment just allocated. If unsuccessful, it returns `NULL`.

The return type `malloc` is `void*` i.e. a pointer of an unknown type. That is because `malloc` does not know what will this memory be used for. The calling function can assign this memory address to a pointer of desired type and manipulate the memory via this pointer.

This memory will not be automatically freed. The function `free` is used to return memory back to heap. It takes a pointer containing the address of a memory segment previously allocated by a call to `malloc`. It is the job of the programmer to call `free` when they have no more use of the memory allocated by `malloc`. Failure to do so would result in memory leaks.

```
1 size_t length = livingPeople ();
2 double * largeVec = malloc ( length * sizeof * largeVec );
3 for ( size_t i = 0; i < length ; ++i) {
4     largeVec [i] = 0.0;
5 }
6 ...
7 free ( largeVec );
```

Listing 10: malloc and free

Listing 11: Caption

You'll use makefiles to compile and run your programs. Get in the habit of making at least following rules in your makefile:

- `all`: to build the project and create an executable
- `clean`: to delete any intermediate files
- `run`: to run the executable with proper arguments
- `check`: to run any tests you might have for your program

4 Exercises

1. Write program that take multiple strings as its arguments. It should show their lengths. It should tell how many of them are copies of each other and display the strings in ascending order.
2. Write a program that would take an email address as its argument and parse the string to separately show the different parts of it.
3. Create a new data type `person` which contains a person's name and their 10 digit phone number. Create a 10 element array of this new type, initialize each element with random values and display all the elements in the array.
4. Using the type `person` write code to provide the functionality of a linked list. Your code should have functions to:
 - add an element to a list
 - remove an element from a list
 - return the length of the list
 - given a name, return their phone number from the list